SIMD and Weight-Output reconfigurable 2D systolic array-based AI accelerator and mapping on Cyclone IV GX

Changli Liu, Jiabao Xu, Wenhao Zhao, Xinqi Li, Zijian Li

December 2025

1 Overview

1.1 Part 1. Vanilla Version

We trained a 4-bit quantized VGG16 model using quantization-aware training and achieved over 90% accuracy. One convolution layer (around `features[27]`) was simplified by reducing its input/output channels to eight and removing the batch normalization layer, enabling a direct mapping onto an 8×8 systolic array. The recovered psums after ReLU matched the reference results with an error below 10^{-3}.

On the hardware side, we implemented a compact RTL core consisting of a 2D MAC array, scratchpad SRAMs, L0 broadcast, OFIFO, and an SFU for accumulation and activation. A testbench was developed to verify the full dataflow of this layer, and the simulated outputs aligned with the expected results. The core was then synthesized onto a Cyclone IV GX FPGA, and we obtained the final frequency, power, and efficiency metrics to evaluate the design.

1.2 Part 2. SIMD Extension

1.2.1 2b4b Quantization

The 2b4b quantized model was trained in `project_2b4b.ipynb`, achieving an accuracy of **89.51%**. In this configuration:

- Activation bits are split into two 2-bit streams.
- Weights remain at 4 bits.
- Data is fed through dual ports in `mac_tile` to support temporal packing.

The `mac_tile` logic supports mode switching via a control signal `ctrl`:

```
assign mac_out_comb = (mac_out_1 << 2) +
    ↪ mac_out_0;
assign out_s = ctrl ? (mac_out_comb + c_q) : (
    ↪ mac_out_0 + mac_out_1 + c_q);
```

When `ctrl=1`, it performs combined multiplication; when `ctrl=0`, it sums two separate products.

To accommodate variable bit-width inputs, the L0 buffer and `xmem` were widened to 8 bits. A `genvar` loop reorganizes incoming data into high/low components for parallel processing.

1.2.2 4b4b Quantization

The 4b4b model was trained in `project_4b4b.ipynb`, achieving **91.36%** accuracy. Both activations and weights use 4-bit representations. Since both weight inputs (`w_0`, `w_1`) are identical, they share the same source within the tile.

Data alignment follows an 8-bit boundary to match memory width, ensuring seamless integration with the existing systolic fabric.

1.3 Part 3. Weight/Output Stationary Reconfigurable Design

This part introduces our re-configurable systolic-array design that supports both *Weight-Stationary (WS)* and *Output-Stationary (OS)* dataflows within a unified hardware template. We add a signal *os_en* to control the hardware functionality. The key goal is to reuse as much of the PE (MAC tile), inter-PE connections, and buffering infrastructure as possible, while switching only the data routing and accumulation behaviors via lightweight control.

2 Detailed Experiment

2.1 Part 1. Vanilla Version

Observed Results and Measured Numbers

The 4-bit quantized VGG16 model achieved a top-1 accuracy of 91.4%, which is comparable to the floating-point baseline and demonstrates the effectiveness of quantization-aware training. The numerical quantization error for the selected squeezed layer was extremely small (2.82×10^{-7}), indicating that the reduced 8-channel configuration and ReLU

integration preserved the correctness of psum recovery.

On hardware, the synthesized RTL core successfully mapped onto the Cyclone IV GX FPGA and achieved a maximum frequency of 127.99 MHz. The design exhibited a dynamic power consumption of 34.01 mW and executed 128 operations per cycle, resulting in a throughput of 0.0164 TOPS. The corresponding energy efficiency was measured as 3.80×10^{-9} TOPS/W. Resource utilization remained modest at 11%, confirming that the core occupied only a small portion of the available logic fabric.

Interpretation and Conclusions

Overall, the results show that aggressive 4-bit quantization can be applied to VGG16 with minimal accuracy degradation when quantization-aware training is used. The extremely low quantization error further confirms that reducing the selected convolution layer to eight channels does not adversely impact numerical stability, while enabling a clean mapping to the 8×8 systolic array.

The FPGA measurements indicate that the design operates correctly on hardware and achieves moderate throughput and low power consumption. Although the reported TOPS and TOPS/W are limited by the older Cyclone IV GX process technology and the relatively small MAC array, the low resource usage suggests substantial headroom for scaling up the design. These results demonstrate that the proposed quantized layer and hardware architecture form a feasible and energy-efficient solution for deploying compact CNN accelerators on resource-constrained FPGA platforms.

2.2 Part 2. SIMD

2.2.1 2b4b Configuration

The 2b4b quantization scheme was implemented to explore aggressive activation compression while maintaining acceptable accuracy. The model was trained in `project_2b4b.ipynb`, achieving a top-1 accuracy of **89.51%**. In this configuration, activations are represented using 2 bits while weights use 4 bits, enabling higher data density and reduced memory bandwidth.

At the hardware level, the `mac_tile` module was extended to support dual-bit-width data paths:

- Two separate activation inputs: `in_x_0` and `in_x_1`, each 2 bits wide
- Two weight inputs: `in_w_0` and `in_w_1`, each 4 bits wide
- Data flows from west to east across the array for both activation and weight streams

The output summation logic incorporates a control signal `ctrl` provided by the testbench to switch between operational modes:

```
assign mac_out_comb = (mac_out_1 << 2) +
    ↪ mac_out_0;
assign out_s = ctrl ? (mac_out_comb + c_q) : (
    ↪ mac_out_0 + mac_out_1 + c_q);
```

When `ctrl = 1`, the high 2-bit activation is shifted left by 2 positions and combined with the low bits before accumulation; when `ctrl = 0`, both partial products are summed independently.

To enable simultaneous loading of two distinct signals within a single cycle, the `mac_array` preserves a dual-port interface:

```
input  [row*a_bw-1:0] in_x_0; // Low bits of
    ↪ activations
input  [row*a_bw-1:0] in_x_1; // High bits of
    ↪ activations
input  [row*w_bw-1:0] in_w_0; // Weight input 0
input  [row*w_bw-1:0] in_w_1; // Weight input 1
```

In the `core_let` module, a `genvar` loop reorganizes incoming 8-bit L0 broadcast data into appropriate bit segments for routing to the MAC array:

```
for (i = 0; i < row; i = i + 1) begin :
    ↪ weight_reorg
    assign reorg_w_1[w_bw*(i+1)-1 : w_bw*i] =
        ↪ ctrl==0 ? data_out_l0[l0_bw*(i+1)-1 :
        ↪ l0_bw*i + 4] : data_out_l0[l0_bw*i + 3
        ↪  : l0_bw*i];
    assign reorg_w_0[w_bw*(i+1)-1 : w_bw*i] =
        ↪ ctrl==0 ? data_out_l0[l0_bw*i + 3 :
        ↪ l0_bw*i] : data_out_l0[l0_bw*i + 3 :
        ↪ l0_bw*i];
    assign reorg_x_1[x_bw*(i+1)-1 : x_bw*i] =
        ↪ ctrl==0 ? data_out_l0[l0_bw*i+ 5 :
        ↪ l0_bw*i + 4] : data_out_l0[l0_bw*i+ 3
        ↪ : l0_bw*i + 2];
    assign reorg_x_0[x_bw*(i+1)-1 : x_bw*i] =
        ↪ ctrl==0 ? data_out_l0[l0_bw*i+ 1 :
        ↪ l0_bw*i] : data_out_l0[l0_bw*i+ 1 :
        ↪ l0_bw*i];
end
```

To accommodate the variable-width data flow, the L0 buffer and `xmem` were widened to 8 bits. Zero-padding is applied to align concatenated 2-bit activation pairs into the 4-bit input format expected by the MAC units.

2.2.2 4b4b Configuration

The 4b4b quantized model, trained in `project_4b4b.ipynb`, achieved a higher accuracy of **91.36%**, serving as a more balanced trade-off between precision and efficiency. Both activations and weights are represented using 4 bits.

For the 4b4b mode where both weight inputs to the `mac_tile` are identical, the design ensures consistency by driving both `b_q_0` and `b_q_1` registers from the same source (`x_in_0`). In `core_let`, both `reorg_w_0` and `reorg_w_1` are connected exclusively to the upper 4 bits of the L0 output: `data_out_l0[l0_bw*i + 3 : l0_bw*i]`.

All data generated in `project_4b4b.ipynb` is aligned to 8-bit boundaries to match the native width of `xmem` and L0, simplifying data movement and avoiding misalignment penalties.

2.2.3 Test Procedure and Results

The evaluation workflow includes:

1. Running `project_2b4b.ipynb` and `project_4b4b.ipynb` to generate quantized models (saved in `./result/VGG16_quant2b4b` and `./result/VGG16_quant4b4b`).
2. Verifying generation of six key files: `activation2b4b.txt`, `weight2b4b.txt`, `output2b4b.txt`, `activation4b4b.txt`, `weight4b4b.txt`, and `output4b4b.txt`.
3. Executing `address_gen.ipynb` to produce tiling-aware address maps `address2b4b.txt` and `address4b4b.txt`.
4. Launching simulations via `source ./cmd.sh` with output channel tiling enabled.

Simulation results confirm correct functionality for both configurations. The generated waveform (shown in `core_tb.vcd`) matches expected behavior, validating the temporal tiling mechanism and dataflow integrity. Output channel tiling is managed through the `och_tile` signal, with weight storage ordered as `K_ij → output_tile → Output_Ch (Col) → Input_Ch (Row)` and output stored as `output_tile → nij_o → Output_Ch`, ensuring compatibility with the systolic execution model.

2.3 Part 3. Output Stationary

2.3.1 Unified PE Datapath and Mode Control

Each PE contains (1) a multiplier-adder (MAC) datapath, (2) local registers for input operands and psum, and (3) selectable input/output routing. We introduce a top-level mode signal `os_en` to control the re-configuration:

- **WS mode** (`os_en`=0): weights are held locally (stationary) and activations stream through the array; psums propagate through the array to form outputs.
- **OS mode** (`os_en`=1): outputs (psums) are accumulated locally and forwarded in a *diagonal* pattern across the array; the final results are written into the output buffer (e.g., `c_q`).

2.3.2 Output-Stationary (OS) Dataflow Implementation

In OS mode, the main challenge is to align the movement of partial sums with the 2D spatial layout so that each output element is accumulated across time without excessive off-chip or long-distance communication.

Diagonal psum propagation. We adopt a diagonal psum propagation scheme: each cycle, a PE updates its local psum and forwards the updated value to a neighbor PE along the diagonal direction. This diagonal behavior ensures that the accumulation for each output coordinate follows a deterministic path through the array. The final accumulated results are captured at the array boundary and pushed into the output queue `c_q` for later writeback.

Input FIFO for activation alignment. OS mode requires careful temporal alignment between activation streams and the diagonal psum wavefront. To support this, we introduce an input FIFO (e.g., `ififo`) that buffers and delays activation inputs so that each PE receives the correct activation at the correct cycle. Compared with WS mode, OS mode is more sensitive to cycle-level scheduling; `ififo` decouples the external input timing from the internal diagonal accumulation schedule.

Output buffering. At the end of OS accumulation, results are collected and stored into `c_q`. This queue provides a clean interface between the compute array and the memory system, enabling burst writes and simplifying backpressure handling.

2.3.3 Test Procedure and Results

Simulation results confirm correct functionality for both WS and OS configurations. We first validated the **weight stationary** dataflow, and then validated the **output stationary** dataflow. The output of testbench (shown as `result.png`)The generated waveform (dumped in `core_tb.vcd`) matches the expected behavior cycle-by-cycle, which validates the temporal tiling mechanism and overall dataflow integrity.

WS validation. We verified that the PEs correctly tiles and loads weights into the array. And the output is exactly the same as part 1. This ensures that, across tiles, each PE receives the intended weights at the correct cycle and that weight reuse matches the systolic execution model.

OS validation. After confirming correct WS functionality, we validated psum accumulation inside the PEs. Outputs are produced and stored with the ordering:

$$\texttt{nij\_o} \rightarrow \texttt{Output Ch},$$

which is consistent with the spatial design of the 2D-systoric array. The waveform and output of the testbench show correct enable timing, register updates, indicating that outputs are generated in the expected sequence without cycle misalignment.

2.4 Part4 Alpha

2.4.1 ResNet20 QAT (Alpha 1)

We reconstructed Layer 2 using a customized Squeeze–Target–Expand block, where the central convolutional stage is constrained to 8 channels and implemented without Batch Normalization. As shown in the module printout, the modified block consists of two 3×3 quantized convolution layers (each with 8 input and 8 output channels) separated by a ReLU activation, forming a lightweight structure suitable for mapping onto an 8×8 systolic array.

Under this configuration, the 4-bit activation and 4-bit weight quantized model achieved an accuracy of 89.58%. The numerical quantization error of the reconstructed layer remained very small (1.69×10^{-6}), indicating that the removal of BatchNorm and channel squeezing did not introduce instability in partial-sum recovery.

2.4.2 PE Gating 1 and 2 (Alpha 2 and Alpha 3)

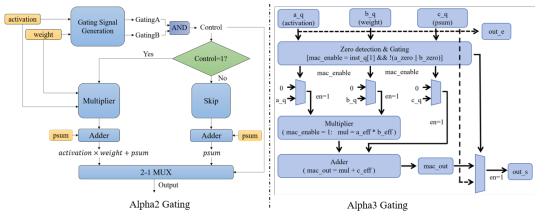

Figure 4: PE Gating Structures

For the first gating design focusing on reducing the number of level switching events at the input part of the multiplier and adder, thereby reducing the overall dynamic power consumption. Only the control signal is 1 can the PE unit calculate and update the value of partial sum. Therefore, the frequency of signal flipping in multiplier and adder is reduced.

The second gating strategy is zero-detection–based operand gating. When either the activation or the weight is zero, the mac-enable signal zeros the multiplier inputs, effectively bypassing the non-effective multiplication, and the accumulated partial sum is directly forwarded to the output.

Data Match	Execute Cycles	Gating Cycles	Gating Ratio
Yes	52	30	57%

Alpha2 Gating Results

	Vanilla (Weight Stationary)	Data_gating
Core Dynamic Thermal Power Dissipation	160.50 mW	142.55 mW

Alpha3 Gating Results

Figure 5: Gatings Results

2.4.3 Output Channel Tiling (Alpha 4)

To handle output channels larger than the systolic array height (8), we implement temporal tiling across output channels. For a total of 16 output channels, data is processed in two tiles of size 8. Key features:

- Weight storage order: `K_ij → output_tile → Output_Ch (Col) → Input_Ch (Row)`
- Output storage order: `output_tile → nij_o → Output_Ch`
- Address generation script: `address_gen.ipynb` generates correct addressing patterns.
- Enabled via testbench control signal `och_tile`.

This approach enables efficient reuse of the fixed-size array without modifying the core datapath.

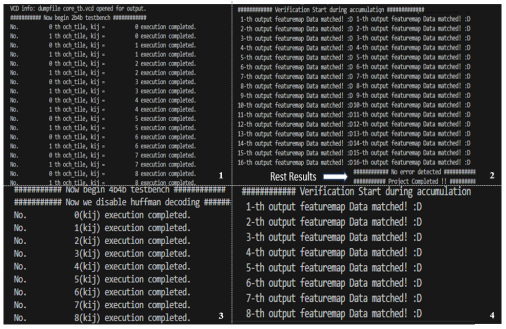

Figure 6: Simulation showing correct temporal tiling behavior across two output channel tiles.

2.4.4 Pruning (Alpha 5)

In this section, we define pruning the model only once before fine-tuning as the one-shot pruning strategy, which will achieve the pruning amount in one go. And we define pruning the model several times during the fine-tuning period as the gradual pruning strategy, which will gradually achieve the pruning amount during the fine-tune period. We compared these two strategies performances based on the unstructured pruning method. To ensure fairness, the fine-tuning epochs for all experiments were set to 10.

Model	Accuracy	Accuracy decline	Pruning Rate	Fine-Tune Epochs
VGG Baseline	88.670%	0.00%	None	None
One-shot Pruning	87.230%	1.44%	0.8	10
One-shot Pruning	84.940%	3.74%	0.8	None
One-shot Pruning	**88.160%**	**0.51%**	**0.9**	**10**
One-shot Pruning	54.160%	34.51%	0.9	None
Gradual Pruning-2times	82.420%	6.25%	0.8	10
Gradual Pruning-4times	82.750%	5.92%	0.8	10
Gradual Pruning-3times	61.430%	27.24%	0.9	10
Gradual Pruning-6times	70.330%	18.34%	0.9	10

Figure 7: Different Unstructured Pruning Results

The reason why the gradual pruning strategy does not achieve the performance as good as the one-shot pruning strategy's may be the pruning frequency during the fine-tuning period. For example, gradual pruning-2 times means that after five epochs, one unstructured pruning will be applied on the model. Therefore, due to the limit fine-tune epochs,

the model may not learn better parameters, leading to model's wrong sparse structure.

2.4.5 Testbench Dual Code (Alpha 6)

Dual core is implemented by instantiating two identical core modules operating in parallel under a shared instruction stream. Both cores execute the same control flow synchronously, and their outputs are concatenated to form a wider channel.

2.4.6 Huffman Encoder (Alpha 7)

The Huffman encoder is designed to exploit the sparsity of quantized activation data, particularly in low-bitwidth configurations such as 4b4b. The encoding policy is as follows:

Zero-value encoding: Input value `0` is encoded using a single bit `'0'`, taking advantage of frequent zero activations.

Non-zero encoding: Any non-zero value is prefixed with a start bit `'1'`, followed by its full 8-bit unsigned representation, resulting in a fixed 9-bit codeword.

The encoding logic is implemented in `Huffman.ipynb`, which reads raw activation data (e.g., from `activation4b4b.txt`) and generates compressed bitstream outputs stored in `activation4b4b_huffman.txt`. Compression performance on representative activation data shows significant reduction in memory footprint:

- Original size: 2304 bits
- Compressed size: 560 bits
- Achieved compression ratio: **4.11**×

This translates directly into reduced off-chip memory bandwidth and lower SRAM storage requirements, improving overall system energy efficiency. The encoded bitstream is fed serially into the Hardware during inference via the `huffman_data_in` port when Huffman mode is enabled.

2.4.7 Huffman Decoder (Alpha 8)

The Huffman decoder hardware structure is shown in the figure below.

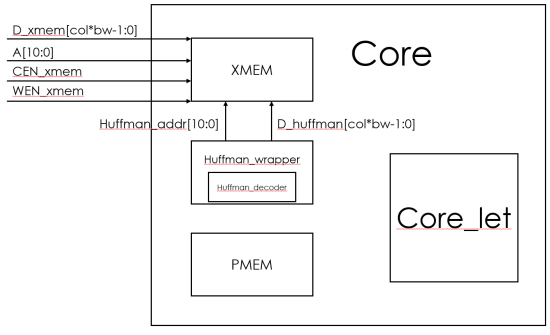

Figure 8: Hardware Structure of Huffman Decoder.

The Huffman decoder consists of two tightly integrated modules that reconstruct compressed activation data on-the-fly:

1. `huffman_decoder.v`: This module receives the serial bitstream and uses a 9-state finite state machine (FSM) to detect the prefix bit and extract the corresponding value:
   - If the incoming bit is `'0'`, it immediately outputs an 8-bit zero and marks the result as valid.
   - If the bit is `'1'`, it proceeds to collect the next 8 bits as the payload, forming a complete non-zero output.

   The FSM ensures correct synchronization and outputs one decoded 8-bit value per valid codeword.

2. `huffman_wrapper.v`: This wrapper buffers eight consecutive decoded values from the decoder and concatenates them into a 64-bit wide word suitable for burst writing into `xmem`. It includes:
   - An 8-state FSM to manage buffer filling and flushing
   - An internal address counter that auto-increments after each 64-bit write
   - Generation of `data_out_valid`, `CEN_xmem`, and `WEN_xmem` control signals

Integration into the core is achieved through dynamic data path switching controlled by instruction bit `inst[5]`:

```
assign xmem_input_data = (inst[5]) ?
    ↪ huffman_data_out : d_xmem;
assign xmem_address = (inst[5]) ? huffman_address
    ↪  : inst[17:7];
assign CEN_xmem = (inst[5]) ? (~
    ↪ huffman_data_out_valid) : inst[19];
assign WEN_xmem = (inst[5]) ? (~
    ↪ huffman_data_out_valid) : inst[18];
```

When `inst[5] = 1`, the core operates in Huffman mode, where data, address, and memory enables are fully managed by the decoder hardware—no software intervention is required.

The successful test waveform (shown in `core_tb.vcd` and illustrated in `huffman_decoder.png`) confirms correct end-to-end functionality, validating the integration of lossless compression into the systolic dataflow.

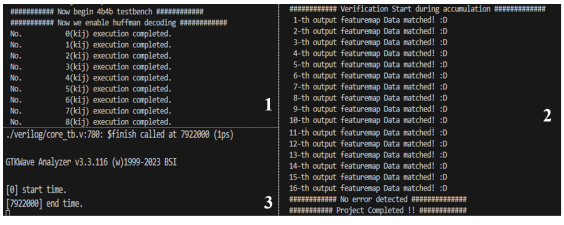

Figure 9: Result of Huffman decoder operation.